

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Constraint Based Problem Solving
Weightage:	40%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	LOKESH S	Reg. No.:	192572151
Section / Batch:	CSE – AI /2 ND YEAR	Date:	25/08/2026

Instructions to Students

- Identify the relevant concepts of I/O interfaces, interrupts, DMA, bus arbitration, and high-speed communication protocols.
- Analyze the I/O requirements considering CPU utilization, latency, throughput, bandwidth, and device priorities.
- Apply suitable I/O techniques such as DMA, vectored interrupts, memory-mapped I/O, nested interrupts, and bus arbitration.
- Compute performance measures such as data transfer rate, interrupt latency, CPU utilization, throughput, and transfer time.
- Interpret the results by evaluating CPU efficiency, communication performance, interrupt response, and suitability of the proposed I/O architecture.

QUESTIONS

1. Design an I/O communication mechanism for a smart agriculture system with the following constraints:
 - 50 field sensors transmit readings at different intervals.
 - Soil-moisture alerts must be processed within 20 μ s.
 - Use interrupt-driven I/O for critical events.
 - Use DMA for periodic bulk sensor-data transfer.
2. Construct an I/O subsystem for a video surveillance system with the following constraints:
 - 8 cameras continuously generate video streams.
 - The aggregate data rate is 1.5 GB/s.
 - Minimize CPU involvement during video transfer.
 - Use DMA with buffer management to prevent data loss.
3. Design a peripheral communication architecture for an industrial automation system with the following constraints:
 - 12 peripherals operate at different data rates.
 - High-priority control devices require deterministic response.
 - Low-priority devices must not monopolize the communication bus.
 - Implement priority-based bus arbitration.
4. Construct a data transfer mechanism for a real-time audio processing system with the following constraints:
 - Audio data must be transferred continuously at 200 MB/s.
 - Transfer interruptions must be minimized.
 - Use double buffering for continuous data processing.
 - Generate an interrupt only when a buffer transfer is completed.
5. Design an interrupt management system for a multicore embedded processor with the following constraints:
 - Support multiple interrupt sources simultaneously.
 - Distribute interrupts across processor cores to balance the workload.
 - Critical interrupts must have higher priority than routine interrupts.
 - Prevent excessive interrupt overhead during high system activity.

Code of Conduct

I LOKESH S (192572151) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: LOKESH S

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Q. No	Problem Understanding (25)	Constraint Analysis (25)	Solution Development (25)	Application of Concepts (15)	Justification (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/25	/ 25	/ 15	/ 10	/ 100

Course Coordinator

Faculty Incharge

1. Smart Agriculture System — I/O Communication Mechanism

The system services 50 field sensors with heterogeneous sampling intervals while guaranteeing that soil-moisture alerts are handled within a 20 μ s bound. A hybrid I/O strategy is used: interrupt-driven I/O for latency-critical events, and DMA for routine bulk transfer.

1.1 Architecture Overview

- Sensor layer: 50 sensors (soil moisture, temperature, humidity, light) connected through analog multiplexers/ADCs to a sensor-bus controller (I2C/SPI aggregation front-end).
- Two parallel data paths from the sensor-bus controller into the MCU: a Critical-Event path (interrupt-driven) and a Bulk-Data path (DMA-driven).
- A Programmable Interrupt Controller (PIC) with priority levels separates the two paths so bulk traffic can never delay a critical alert.

1.2 Interrupt-Driven Path for Soil-Moisture Alerts

- Each soil-moisture sensor has a threshold comparator; crossing the threshold asserts a dedicated, edge-triggered interrupt line rather than requiring polling.
- The alert interrupt is assigned the highest priority level in the PIC, above all bulk-transfer and housekeeping interrupts, and is configured as non-maskable within the sensing subsystem.
- The Interrupt Service Routine (ISR) is kept minimal: it reads the sensor ID and raw value directly from a memory-mapped register (no bus renegotiation), sets an alert flag, and returns — deferred actions (logging, actuator commands) run in a low-priority background task.
- Interrupt latency budget (must fit inside 20 μ s): interrupt detection $\approx$ 1–2 μ s, context save $\approx$ 3–5 μ s, ISR execution $\approx$ 5–8 μ s, register read/flag-set $\approx$ 2–3 μ s — total well under the 20 μ s bound with margin for jitter.
- Nested interrupts are permitted only for equal-or-higher-priority alerts; lower-priority DMA-complete interrupts cannot preempt an in-progress alert ISR.

1.3 DMA Path for Periodic Bulk Sensor Data

- A timer-driven DMA controller polls the remaining 49 non-alert channels on a fixed schedule (e.g., every 500 ms–5 s, matched to each sensor's required interval) using a scan-list of source addresses.
- Each DMA cycle transfers a full sweep of sensor readings directly into a circular memory buffer without CPU intervention; the CPU is interrupted only once per completed sweep, not once per sensor.
- Double-buffered DMA descriptors let the next sweep begin filling buffer B while buffer A is drained by the application/logging task, avoiding stalls.

1.4 Priority & Timing Summary

Path	Trigger	Mechanism	Priority	Target Latency
Soil-moisture alert	Threshold crossing	Dedicated interrupt + minimal ISR	Highest (P0)	< 20 μ s
Other critical sensors	Threshold/fault	Interrupt, shared handler	High (P1)	< 100 μ s

Path	Trigger	Mechanism	Priority	Target Latency
Bulk periodic sensing	Timer (scan-list)	DMA sweep, buffer-complete IRQ	Low (P2)	Best-effort / seconds

2. Video Surveillance System — I/O Subsystem

Eight cameras generate a combined 1.5 GB/s (≈ 187.5 MB/s per camera). At this rate, CPU-mediated (programmed I/O) transfer is infeasible; the subsystem is built around scatter-gather DMA with managed circular buffers per camera.

2.1 Data Path

- Each camera's video interface (e.g., MIPI-CSI/PCIe capture front-end) writes directly into a per-camera DMA engine — no CPU copy in the capture path.
- A multi-channel DMA controller with 8 independent channels (one per camera) allows concurrent, hardware-parallel transfer, avoiding channel contention.
- Scatter-gather descriptor chains let each DMA channel write into a set of pre-allocated, non-contiguous frame buffers, so buffer memory does not need to be physically contiguous.

2.2 Buffer Management (loss prevention)

- Per-camera circular buffer pool ($N \geq 3$ buffers, e.g., triple-buffered) so one buffer is being filled by DMA, one is being drained/encoded, and one is held as slack for scheduling jitter.
- Producer–consumer synchronization: DMA (producer) advances a write pointer; the video-processing/encoding task (consumer) advances a read pointer; buffers are recycled only after the consumer confirms completion.
- Backpressure / overflow policy: if the consumer falls behind, the system either (a) drops the oldest unconsumed frame with an overrun counter, or (b) throttles the affected camera's capture rate — both configurable per deployment, but the DMA ring itself is never overwritten out of order.
- Watermark interrupts: a near-full watermark on each ring triggers an early, low-priority interrupt to schedule the consumer before hard overflow occurs.

2.3 Minimizing CPU Involvement

- CPU touches data only at buffer boundaries: one interrupt per completed buffer (not per packet/frame line), i.e., interrupt coalescing.
- Interrupt handling is deferred to a lightweight bottom-half/tasklet that just marks the buffer ready; heavier work (compression, storage, streaming) runs in a separate worker thread outside interrupt context.
- Where supported, DMA engines aggregate completions across channels into a single status register poll/interrupt, further reducing IRQ rate under 8-camera concurrent load.

2.4 Bandwidth & Buffer Sizing

Parameter	Value
Cameras	8
Aggregate throughput	1.5 GB/s
Per-camera throughput	≈ 187.5 MB/s
DMA channels	8 (one per camera, parallel)

Parameter	Value
Buffers per camera	3 (triple buffering)
CPU interaction	Buffer-complete interrupt only

3. Industrial Automation System — Peripheral Communication Architecture

Twelve peripherals operate at different rates on a shared communication bus. High-priority control devices (e.g., safety interlocks, motor controllers) need deterministic, bounded-latency access; low-priority devices (e.g., status displays, logging sensors) must never be able to stall the bus.

3.1 Priority-Based Bus Arbitration

- Peripherals are grouped into three priority classes: Critical-Control (deterministic), Standard-Control (soft real-time), and Monitoring/Logging (best-effort).
- A priority arbiter (comparable to CAN-style bitwise arbitration or a fixed-priority bus grant scheme) grants bus ownership to the highest-priority pending request each arbitration cycle.
- Critical-Control devices are given reserved, guaranteed time slots (a TDMA-style micro-schedule overlaid on the arbitration) so their maximum wait time is bounded regardless of other traffic — this is what makes the response deterministic, not just "usually fast".

3.2 Preventing Low-Priority Monopolization

- A bus-hold time limit (maximum burst length) is enforced per transaction; no device — including high priority ones — can hold the bus indefinitely.
- An aging/anti-starvation mechanism raises the effective priority of a low-priority request the longer it waits, guaranteeing it eventually wins arbitration even under continuous high-priority traffic.
- A minimum guaranteed bandwidth allocation (time-slice quota) is reserved for the Monitoring/Logging class within each arbitration super-cycle.

3.3 Priority Table (example allocation for 12 peripherals)

Class	Example Devices	Count	Arbitration	Latency Goal
Critical-Control	Safety interlock, emergency stop, motor drive	3	Reserved TDMA slot + highest fixed priority	Deterministic, μ s-level
Standard-Control	Valve actuator, conveyor controller, PID loop I/O	4	Fixed priority with bus-hold cap	ms-level, bounded
Monitoring/Logging	Temperature log, HMI display, status LED, data logger	5	Round-robin with aging + min. bandwidth quota	Best-effort, no starvation

3.4 Summary

- Determinism for critical devices comes from reserved slots, not just "higher priority" alone.
- Starvation of low-priority devices is prevented by aging plus a guaranteed minimum bandwidth quota, and no device can monopolize the bus due to the bus-hold time cap.
-

4. Real-Time Audio Processing System — Data Transfer Mechanism

Continuous audio at 200 MB/s cannot tolerate transfer gaps (audible glitches/underruns). The mechanism uses DMA-driven double buffering so the CPU/DSP is only interrupted once per buffer, never mid-stream.

4.1 Double-Buffering Design

- Two equally-sized buffers (Buffer A, Buffer B) are allocated for the audio stream (ping-pong buffering).
- The DMA controller fills Buffer A from the ADC/audio codec while the DSP/CPU simultaneously processes the already-full Buffer B (filter, encode, mix, etc.).
- When Buffer A finishes filling, the DMA controller automatically switches to filling Buffer B (via a chained/circular DMA descriptor) — no CPU action is required to keep the stream flowing, so there is no gap between buffers.
- Roles swap continuously: A fills → B processes → B fills → A processes → ... this cycle repeats for the life of the stream.

4.2 Interrupt Policy

- The DMA raises exactly one interrupt per completed buffer transfer (a "buffer-complete" interrupt) — there is no per-sample or per-packet interrupt, which is essential at 200 MB/s to avoid overwhelming the CPU.
- The buffer-complete ISR does minimal work: it swaps the active/processing buffer pointers and signals the audio-processing task; the actual DSP work happens outside interrupt context so the ISR itself stays short and predictable.
- Buffer size is chosen to balance interrupt rate against latency, e.g., a buffer sized for ~1–5 ms of audio at 200 MB/s gives an interrupt roughly every 1–5 ms — frequent enough for low latency, infrequent enough to keep CPU overhead low.

4.3 Why This Minimizes Interruptions

- Because DMA autonomously continues into the second buffer the instant the first completes, there is no window where the transfer stalls waiting on the CPU.
- The CPU's only job is to consume a buffer before the DMA wraps back around to it — sized correctly, this leaves comfortable slack and eliminates underrun/overrun glitches.

5. Multicore Embedded Processor — Interrupt Management System

The system must handle many simultaneous interrupt sources, spread the handling load across cores, guarantee that critical interrupts preempt routine ones, and avoid interrupt storms degrading throughput under heavy load.

5.1 Distribution Across Cores

- A hardware Interrupt Distributor (analogous to an ARM GIC) sits between interrupt sources and the per-core interrupt controllers, and can route any interrupt to any core or core group.
- Load balancing policy: routine/general interrupts are distributed round-robin (or based on current per-core load) across available cores so no single core becomes an interrupt bottleneck.
- Affinity pinning: latency-critical interrupt sources can be pinned (targeted) to a dedicated core that is kept lightly loaded, ensuring predictable response for that source regardless of what other cores are doing.

5.2 Priority Scheme

- Interrupts are classified into priority tiers (e.g., Critical / High / Routine / Background); the distributor enforces that a Critical interrupt preempts an in-progress Routine ISR on its target core.
- Priority masking (threshold registers) lets each core temporarily block lower-priority interrupts while servicing a higher-priority one, preventing priority inversion.
- Critical interrupts are additionally given a reserved core (or the least-loaded core) so they are never queued behind a backlog of routine work.

5.3 Controlling Overhead Under High Activity

- Interrupt coalescing/batching: bursts of routine interrupts from the same source are merged into a single delivery where the hardware/driver supports it, cutting the per-event overhead.
- Threaded/deferred interrupt handling: the hardware ISR does the minimum (acknowledge + queue), and the bulk of the work runs in a schedulable bottom-half/softIRQ thread — this bounds time spent in interrupt context even when many interrupts fire.
- Adaptive polling fallback: if interrupt rate for a given routine source exceeds a threshold (an "interrupt storm"), the system temporarily switches that source to polled mode until the rate subsides, then reverts to interrupt-driven mode — this bounds worst-case overhead while keeping normal-load latency low.
- Rate limiting / interrupt moderation timers cap how often a single low-priority source can interrupt any core per unit time, protecting overall system throughput.

5.4 Summary Table

Requirement	Mechanism
Multiple simultaneous sources	Central interrupt distributor routes to any core
Load balancing across cores	Round-robin / load-aware routing for routine interrupts
Critical > routine priority	Priority tiers + preemption + priority masking

Requirement	Mechanism
Bounded overhead under load	Coalescing, threaded bottom-halves, adaptive polling fallback, rate limiting